МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ» (ДГТУ)

Факультет
Информатика и вычислительная техника

Кафедра
Программное обеспечение вычислительной техники и автоматизированных систем

Создание многослойной нейронной сети прямого распространения с алгоритмом обучения «Обратное распространение ошибок»

Практикум

по выполнению лабораторной работы №1
по дисциплине «Нейронные сети и глубокое обучение»

Большие данные и машинное обучение
(указывается направленность (профиль) образовательной программы)

02.03.03 Математическое обеспечение и администрирование информационных систем
(указывается код и наименование направления подготовки)

Ростов-на-Дону
2026 г.

Составители: канд. техн. наук, доц. каф. А.А. Скляренко

УДК 004.432

Многослойная нейронная сеть: метод. указания. – Ростов н/Д: Издательский центр ДГТУ, 2026.

В методическом указании рассматриваются вопросы создания многослойных сетей прямого распространения и алгоритма их обучения, основанного на обратном распространении ошибок. Приведены задания к лабораторной работе, помогающие закрепить на практике полученные знания, и контрольные вопросы для самопроверки. Предназначено для обучающихся по направлению 02.03.02 «Математическое обеспечение и администрирование информационных систем», профиль «Большие данные и машинное обучение».

Ответственный за выпуск:
зав. кафедрой «Программное обеспечение вычислительной техники и автоматизированных систем» Долгов В.В.

© А.А. Скляренко,
2026
© Издательский центр ДГТУ, 2026

1 Теоретическая часть

1.1 Полносвязная нейронная сеть прямого распространения

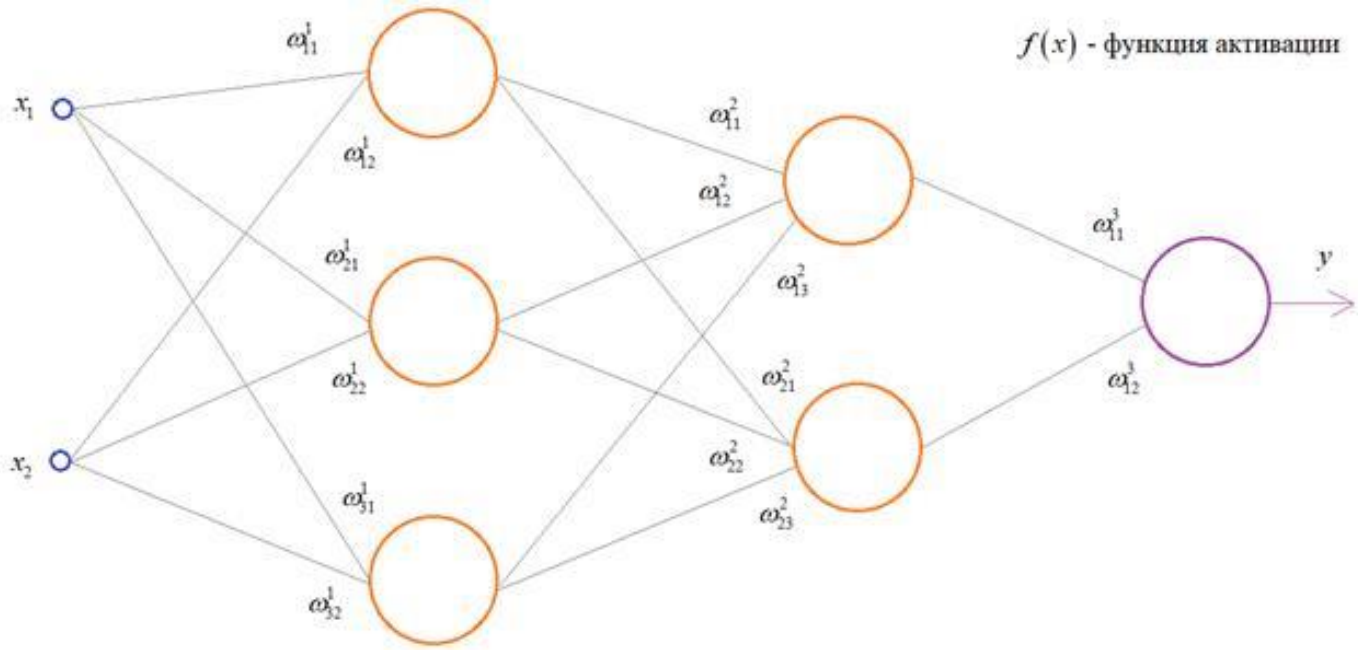
Полносвязная нейронная сеть (сеть прямого распространения, Feedforward Neural Networks, FNN) — это простая форма нейронных сетей, в которой информация движется только в одном направлении — от входного слоя к выходному.

Такие сети состоят из множества скрытых слоёв, где каждый нейрон соединён со всеми нейронами предыдущего и следующего слоя.

Обычно полносвязные нейронные сети используются для задач классификации и регрессии.

Далее рассматривается нейронная сеть для решения задачи регрессии или бинарной классификации.

Рассмотрим полносвязная НС прямого распространения, представленную на рисунке, с весами связей, выбранными произвольным образом в диапазоне от [-0.5; 0,5]. Здесь верхний индекс показывает принадлежность к тому или иному слою сети. Также, каждый нейрон имеет некоторую активационную функцию $f(x)$:



На первом шаге делается прямой проход по сети. Мы пропускаем вектор наблюдения $[x_1, x_2]$ через эту сеть, и запоминаем все выходные значения нейронов скрытых слоев:

$$f_{11} = f\left(w_{11}^1 x_1 + w_{12}^1 x_2\right)$$
$$f_{12} = f\left(w_{21}^1 x_1 + w_{22}^1 x_2\right) \qquad f_{21} = f\left(w_{11}^2 f_{11} + w_{12}^2 f_{12} + w_{13}^2 f_{13}\right)$$
$$f_{13} = f\left(w_{31}^1 x_1 + w_{32}^1 x_2\right) \qquad f_{22} = f\left(w_{21}^2 f_{11} + w_{22}^2 f_{12} + w_{23}^2 f_{13}\right)$$

и последнее выходное значение y:

$$v_{out} = w_{11}^3 \cdot f_{21} + w_{12}^3 \cdot f_{22}$$
$$y = f\left(v_{out}\right)$$

Далее, мы знаем требуемый отклик d для текущего вектора $[x_1, x_2]$, значит для него можно вычислить ошибку работы НС. Она будет равна:

$$e = y - d$$

Дальше начинается самое главное – корректировка весов. Для этого делается обратный проход по НС: от последнего слоя – к первому.

Итак, у нас есть ошибка e и некая функция активации нейронов $f(x)$. Первое, что нам нужно – это вычислить локальный градиент для выходного нейрона. Это делается по формуле:

$$\delta = e \cdot f'\left(v_{out}\right)$$

В качестве функции активации для сетей с небольшим числом слоев, часто применяют или гиперболический тангенс:

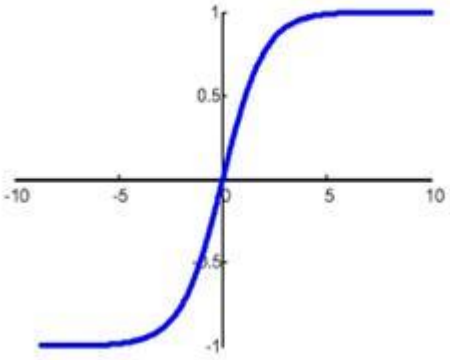
$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

или логистическую функцию:

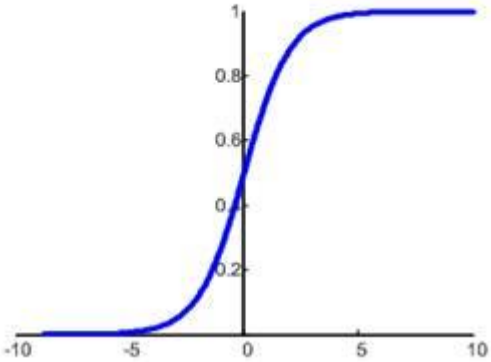
$$f(x) = \frac{1}{1 + e^x}$$

Фактически, они отличаются только тем, что первая дает выходной интервал [-1; 1], а вторая – [0; 1]. И мы уже берем ту, которая нас больше устраивает в данной конкретной ситуации. Например, выберем логистическую функцию.

гиперболический тангенс



логистическая функция



Эти функции могут быть применены для решения задачи бинарной классификации, сравнивая выходное значение нейронной сети с порогом 0,5 (или 0), если порог превышен 1 класс, иначе 2 класс.

Ее производная функции по аргументу x дает очень простое выражение:

$$f'(x) = f(x) \cdot (1 - f(x))$$

Именно его мы и запишем в нашу формулу вычисления локального градиента:

$$\delta = e \cdot f'(v_{out}) = e \cdot f(v_{out}) \cdot (1 - f(v_{out}))$$

Но, так как

$$y = f(v_{out})$$

то локальный градиент последнего нейрона, равен:

$$\delta = e \cdot y \cdot (1 - y)$$

Отлично, это сделали. Теперь у нас есть все, чтобы выполнить коррекцию весов. Начнем со связи w_{11}^3, формула будет такой:

$$w_{11}^3 = w_{11}^3 - \begin{bmatrix} \lambda \\ шаг \\ сходимости \end{bmatrix} \cdot \begin{bmatrix} \delta \\ локальный \\ градиент \end{bmatrix} \cdot \begin{bmatrix} f_{21} \\ выходной \\ сигнал \end{bmatrix}$$

Для второй связи все то же самое, только входной сигнал берется от второго нейрона:

$$w_{12}^3 = w_{12}^3 - \begin{bmatrix} \lambda \\ шаг \\ сходимости \end{bmatrix} \cdot \begin{bmatrix} \delta \\ локальный \\ градиент \end{bmatrix} \cdot \begin{bmatrix} f_{22} \\ выходной \\ сигнал \end{bmatrix}$$

Параметр λ (скорость обучения) подбирается самостоятельно, вручную самим разработчиком. В самом простом случае можно попробовать следующие значения:

$$\lambda = 0,1;\ 0,01;\ 0,001$$

Итак, мы с вами скорректировали связи последнего слоя. Переходим к нейрону следующего с конца слоя и для его входящих связей повторим ту же саму процедуру. Но для этого, нужно знать значение его локального градиента. Определяется он просто. Локальный градиент последнего нейрона взвешивается весами входящих в него связей. Полученные значения на каждом нейроне умножаются на производную функции активации, взятую в точках входной суммы:

$$\delta_{21} = \delta \cdot w_{11}^3 \cdot f'(v_{21}) = \delta \cdot w_{11}^3 \cdot \left[f_{21} \cdot (1 - f_{21}) \right]$$
$$\delta_{22} = \delta \cdot w_{12}^3 \cdot f'(v_{22}) = \delta \cdot w_{12}^3 \cdot \left[f_{22} \cdot (1 - f_{22}) \right]$$

А дальше действуем по такой же самой схеме, корректируем входные связи по той же формуле:

$$w_{11}^2 = w_{11}^2 - \lambda \cdot \delta_{21} \cdot f_{11}$$
$$w_{12}^2 = w_{12}^2 - \lambda \cdot \delta_{21} \cdot f_{12}$$
$$w_{13}^2 = w_{13}^2 - \lambda \cdot \delta_{21} \cdot f_{13}$$

И для второго нейрона:

$$w_{21}^2 = w_{21}^2 - \lambda \cdot \delta_{22} \cdot f_{11}$$
$$w_{22}^2 = w_{22}^2 - \lambda \cdot \delta_{22} \cdot f_{12}$$
$$w_{23}^2 = w_{23}^2 - \lambda \cdot \delta_{22} \cdot f_{13}$$

Осталось скорректировать веса первого слоя. Снова вычисляем локальные градиенты для нейронов первого слоя, но так как каждый из них имеет два выхода, то сначала вычисляем сумму от каждого выхода:

$$\sigma_{11} = \delta_{21} \cdot w_{11}^2 + \delta_{22} \cdot w_{21}^2$$
$$\sigma_{12} = \delta_{21} \cdot w_{12}^2 + \delta_{22} \cdot w_{22}^2$$
$$\sigma_{13} = \delta_{21} \cdot w_{13}^2 + \delta_{22} \cdot w_{23}^2$$

А затем, значения локальных градиентов на нейронах первого скрытого слоя:

$$\delta_{11} = \sigma_{11} \cdot f'(v_{11}) = \sigma_{11} \cdot \left[f_{11} \cdot (1 - f_{11}) \right]$$
$$\delta_{12} = \sigma_{12} \cdot f'(v_{12}) = \sigma_{12} \cdot \left[f_{12} \cdot (1 - f_{12}) \right]$$
$$\delta_{13} = \sigma_{13} \cdot f'(v_{13}) = \sigma_{13} \cdot \left[f_{13} \cdot (1 - f_{13}) \right]$$

Ну и осталось выполнить коррекцию весов первого слоя все по той же формуле:

$$w_{11}^1 = w_{11}^1 - \lambda \cdot \delta_{11} \cdot x_1$$
$$w_{12}^1 = w_{12}^1 - \lambda \cdot \delta_{11} \cdot x_2$$

$$w_{21}^1 = w_{21}^1 - \lambda \cdot \delta_{12} \cdot x_1$$
$$w_{22}^1 = w_{22}^1 - \lambda \cdot \delta_{12} \cdot x_2$$

$$w_{31}^1 = w_{31}^1 - \lambda \cdot \delta_{13} \cdot x_1$$
$$w_{32}^1 = w_{32}^1 - \lambda \cdot \delta_{13} \cdot x_2$$

В результате, мы выполнили одну итерацию алгоритма обучения НС. На следующей итерации мы должны взять другой входной вектор из нашего обучающего множества. Лучше всего это сделать случайным образом, чтобы не формировались возможные ложные закономерности в последовательности данных при обучении НС. Повторяя много раз этот процесс, весовые связи будут все точнее описывать обучающую выборку.

Далее необходимо определить критерий качества, который бы минимизировался алгоритмом градиентного спуска. В действительности, мы стремились получить минимум суммы квадратов ошибок для обучающей выборки:

$$E = \frac{1}{2} \sum_{j=1}^{N} e_j^2 = \frac{1}{2} \sum_{j=1}^{N} \left(d_j - y_j \right)^2$$

То есть, с помощью алгоритма градиентного спуска веса корректируются так, чтобы минимизировать этот критерий качества работы НС.

1.2 Полносвязная нейронная сеть на основе фреймворка TensorFlow + API Keras

TensorFlow — это платформа с открытым исходным кодом для построения и обучения моделей машинного и глубокого обучения, разработанная компанией Google.

Keras — это высокоуровневое API для работы с нейронными сетями, которое работает поверх

Основные компоненты полносвязной нейронной сети:

- Входной слой
- Скрытые слои
- Выходной слой
- Функции активации
- Функция потерь
- Оптимизатор

Порядок создания многослойной сети:

1. Подключить необходимые модули:

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
```

2. Подготовка данных:

- Загрузить и предобработать данные
- Разделить на обучающую и тестовую выборки
- Нормализовать данные

```
# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Нормализация данных
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

python

```
# Пример загрузки данных
X_train = ...  # входные данные
y_train = ...  # целевые значения
```

При решении задачи классификации также необходимо выполнить кодирование целевого значения на основе one-hot encoding.

```
y_train = to_categorical(y_train, num_classes=3)

y_test = to_categorical(y_test, num_classes=3)
```

3. Создание модели:

```
python
# Создаем последовательную модель
model = Sequential()

# Добавляем слои
model.add(Dense(64, input_dim=X_train.shape[1],
activation='relu'))  # входной слой
model.add(Dense(32, activation='relu'))  # скрытый слой
model.add(Dropout(0.3))
model.add(Dense(1, activation='sigmoid'))  # выходной слой
```

Слой Dropout – это метод регуляризации для борьбы с переобучением. Случайным образом отключает часть нейронов в соответствии с процентом, который указан как параметр, т.е. в примере будет отключено 30% нейронов.

Выходной слой нейронной сети будет состоять из одного нейрона, если решается задача регрессии или бинарной классификации (выходное значение сравнивается с порогом 0.5 и объект относится или к 1 или к 2 классу).

При решении задачи регрессии для нейрона выходного слоя не указывается функция активации.

При решении многоклассовой классификации выходной слой состоит из количества нейронов равных количеству классов и имеет функцию активации softmax.

```
model.add(Dense(3, activation='softmax'))  # выходной слой
```

4. Компиляция модели:

При решении бинарной классификации указываются следующие параметры модели:

```
model.compile(optimizer=Adam(learning_rate=0.001),
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

При решении многоклассовой классификации указываются следующие параметры модели:

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

При решении задачи регрессии указываются следующие параметры:

```
model.compile(optimizer='adam',
              loss='mean_squared_error',
              metrics=['mean_absolute_error'])
```

5. Обучение модели:

python

```
# Обучение модели
history = model.fit(X_train, y_train,
                    epochs=50,
                    batch_size=32,
                    validation_split=0.2)
```

Объект history содержит следующие поля:

- history[‘loss’] – значения функции потерь на этапе обучения
- history[‘val_loss’] – значение функции потерь на этапе валидации
- history[‘accuracy’] – значения точности на этапе обучения
- history[‘val_accuracy’] – значение точности на этапе валидации
- history['mean_absolute_error'] – значение абсолютной ошибки на этапе обучения
- history['val_mean_absolute_error'] – значение абсолютной ошибки на этапе валидации

6. Оценка качества:

python

```
# Оценка на тестовой выборке для решения задачи классификации
loss, accuracy = model.evaluate(X_test, y_test)
```

```
print(f'Точность: {accuracy:.4f}')

# Оценка на тестовой выборке для решения задачи регрессии
test_loss, test_mae = model.evaluate(X_test, y_test)
print(f'MSE на тестовых данных: {test_loss:.4f}')
print(f'MAE на тестовых данных: {test_mae:.4f}')
```

8. Визуализация динамики обучения нейронной сети на основе данных из объекта history.

```
# График обучения
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Loss на обучении')
plt.plot(history.history['val_loss'], label='Loss на валидации')
plt.title('Функция потерь')
plt.xlabel('Эпохи')
plt.ylabel('MSE')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['mean_absolute_error'], label='MAE на
обучении')
plt.plot(history.history['val_mean_absolute_error'], label='MAE на
валидации')
plt.title('Средняя абсолютная ошибка')
plt.xlabel('Эпохи')
plt.ylabel('MAE')
plt.legend()


    plt.show()
```

9. Предсказание

```
# Предсказания
predictions = model.predict(X_test)
```

2 Задание к лабораторной работе

Задание 1

Написать код для реализации многослойной сети прямого распространения, не используя библиотеки и фреймворки для машинного и глубокого обучения.

В структуре кода необходимо реализовать следующие классы:

1. Класс «FunActivation» - содержит методы с функцией активацией сигмоид и метод с ее производной.
2. Класс «Neiron» - содержит веса входящих в него связей и метод вычисляющей выходное значение нейрона при прямом проходе
3. Класс «DenseLayer» (слой) – содержит массив нейронов и метод вычисляющий выходной вектор данного слоя
4. Класс «Network» (сеть) – содержит массив слоев, метод вычисляющее выходное значение сети, метод, реализующий обучение нейронной сети по алгоритму обратного распространения ошибки.
5. Создать многослойную нейронную сеть и обучить ее под задачу согласно варианту.

   Из датасета берете только 4 входных, числовых признака. Нормализуете их. Оставляете два класса. Можно уменьшить размер датасета для обучения.
   Если числовых признаков не будет хватать, можно взять категориальные признаки и их закодировать числом.

Вариант	**Задача**
1	Датасет Ирисы: выбрать два класса (0 и 1). Входные признаки обязательно нормализовать. Для обучения можно использовать не весь датасет. Iris Species

2	**Cleveland Heart Disease Dataset**. Содержит 303 записи (пациента) с 13 признаками (возраст, пол, холестерин и др.). Целевая переменная — наличие сердечного заболевания, закодирована от 0 (отсутствие) до 4 (наличие). buio/heart-disease · Datasets at Hugging Face На вход выбрать 4 признака и взять только два класса (0 и 1). Входные признаки обязательно нормализовать.
3	Данные о пингвинах с острова Палмера. На вход выбрать 4 признака и взять только два класса (0 и 1). Входные признаки обязательно нормализовать. penguin dataset : The new Iris
4	Датасет о Вине. На вход выбрать 4 признака и взять только два класса (0 и 1). Входные признаки обязательно нормализовать. Wine Quality Dataset
5	Датасет про диабет. На вход выбрать 4 признака и взять только два класса (0 и 1). Входные признаки обязательно нормализовать. Diabetes Dataset
6	Датасет для классификации **сердечно-сосудистых заболеваний**. На вход выбрать 4 признака и взять только два класса (0 и 1). Входные признаки обязательно нормализовать. Heart Disease Classification
7	Результаты общего анализа крови, предназначенные для определения различных типов **анемии**. На вход выбрать 4 признака и взять только два класса (0 и 1). Входные

	признаки обязательно нормализовать. Anemia Types Classification
8	Car Classification Dataset — Содержит записи с **характеристиками автомобилей** (цена покупки и обслуживания, число дверей, вместимость, размер багажника, уровень безопасности). Целевой признак — **оценка автомобиля** (от «неприемлемо» до «очень хорошо»).
9	Parkinson Dataset — 195 записей о пациентах с болезнью Паркинсона, с 25 параметрами анализов. Можно использовать для предварительной оценки отличия больных людей от здоровых. Parkinson's Disease Data Set
10	Titanic Dataset — содержит информацию про пассажиров (возраст, пол, родственники на борту и пр). Целевая переменная – выжил или не выжил пассажир Titanic Dataset

Возможно рассмотреть свой вариант датасета формата csv для классификации, но в группе не должно быть одинаковых датасетов, выбранных самостоятельно.

Задание 2.

Подобрать архитектуру и реализовать многослойной нейронной сети прямого распространения на основе TensorFlow + Keras для решения задачи классификации по данным из датасета из 1- ого задания. Только теперь рассматриваются все входные признаки и все классы. Произвести обучение модели и последующее тестирование.

3 Материально-техническое обеспечение работы

Аудитория для проведения лабораторных занятий должна быть укомплектована специализированной мебелью и индивидуальными компьютерами следующей минимальной комплектации:

- Процессор: не менее двух исполнительных ядер, совместимый с системой команд x86 и x64, с поддержкой аппаратной виртуализации.
- Оперативная память: не менее 8 Гб.
- Монитор: не менее 24" (дюймов) по диагонали.
- Наличие локальной сети со скоростью обмена не менее 1 Гб/сек.
- Наличие доступа в сеть Интернет со скоростью не менее 1 Мбит/сек.
- Наличие клавиатуры и манипулятора «мышь».

На компьютерах должно быть установлено следующее программное обеспечение:

- Операционная система: Microsoft Windows 10 (или выше) и/или одна из операционных систем семейства Linux, допускающих установку среды исполнения для Python. Допускается конфигурация, когда одна из операционных систем установлена внутри виртуальной машины (гипервизора).
- Среды разработки и исполнения программ: Google Colab, PyCharm, VS Code.

4 Порядок выполнения и сдачи работы

Для выполнения лабораторной работы рекомендуется придерживаться следующего порядка выполнения:

1. Ознакомится с темой и целями лабораторной работы
2. Изучить теоретический материал
3. Подготовить рабочее окружение
4. Разработать программный код лабораторной работы в соответствие с заданием
5. Произвести проверку работоспособности, выполнить тестирование и отладку кода
6. Проанализировать полученные результаты и примененные в ходе решения подходы
7. Выполнить самооценку и рефлексию

8. Сдать лабораторную работу преподавателю и получить от него обратную связь

При сдаче студентом лабораторной работы основным отчетом выступает исходный код самостоятельно созданной в процессе выполнения работы программы.

Исходный код должен быть отформатирован согласно принятым для используемого языка программирования стандартам. Является желательным наличие в исходном коде комментариев, описывающих основные части программы и особенности их функционирования. В то же время студент должен быть готов объяснить работу программы в целом и каждую отдельную ее часть при полном отсутствии комментариев (например, они могут быть удалены преподавателем при сдаче работы).

Обязательным условием сдачи является умение студента самостоятельно восстановить любой участок исходного кода программы (но не более 20 строк подряд) после его удаления. Удаленный участок должен быть самостоятельно набран студентом заново в присутствие преподавателя. При восстановлении участка запрещается использовать операции Undo (Отменить) текстовых редакторов или переписывание кода участка из других источников.

Исходный код сдаваемой программы должен быть представлен в электронном виде.

Сдача исходного кода ранее сдававшихся программ или программ, код которых выложен в сети Интернет, не допускается.

5 Контрольные вопросы к лабораторной работе

1. Что такое нейронная сеть прямого распространения?
2. Какие функции активации используются в нейронных сетях?
3. Для чего нужна функция потерь?
4. Какие оптимизаторы существуют в Keras?
5. Как происходит процесс обучения нейронной сети?

6 Перечень использованных информационных ресурсов

1. Шолле Франсуа Глубокое обучение на Python. — СПб.: Питер, 2018. — 400 с.: ил. — (Серия «Библиотека программиста»).

2. Николенко С., Кадурин А., Архангельская Е. Глубокое обучение. — СПб.: Питер, 2018. — 480 с.: ил. — (Серия «Библиотека программиста»).

Редактор А.А. Литвинова

ЛР № 04779 от 18.05.01.	В набор	В печать
Объем 0,5 усл.п.л., уч.-изд.л.	Офсет.	Формат 60x84/16.
Бумага тип №3.	Заказ №	Тираж 75. Цена

Издательский центр ДГТУ

Адрес университета и полиграфического предприятия:
344010, г. Ростов-на-Дону, пл. Гагарина, 1.